Crew 1.0 - Configuration Reference

This document has two parts. The first is short and hand-written: how crew's two config files combine, how to see what is in force, and the common setups. The second is generated from the code and lists every key.

How the Two Files Combine

crew reads two JSON files:

- `~/.claude/crew/config.json` - the **machine-global** file. Your personal defaults for every repo on this machine: providers and models, notifications, auto-clear, the guards, how far autopilot may go.
- `.crew/config.json` - the **repo** file. Facts about one checkout: the tracker, the board, scope enforcement, production declarations.

The repo file wins over the machine file, which wins over crew's defaults. Four exceptions, each named in the Layer column below:

- A **repo-only** key in the machine file takes effect nowhere. It is pruned on read and refused on write.
- A **ratcheted** key (`install.policy`, each guard `guards.profile` and `guards.overrides` expand to, `change.requireForProduction`, `environments.prodUnattended`) resolves to the narrower of the two values, so a cloned repo cannot widen what your machine allows.
- A **machine-armed** key (`resume.auto`, `context.autoClear.enabled`) can be turned on only in the machine file. A repo can only switch it off.
- A **personal** key (the five `autopilot.*` keys) resolves per key to the stricter of the layers that set it; a layer that says nothing imposes nothing. A machine value is your default for every repo, which a repo may narrow and never widen.

A `null` in the repo file over a machine value inherits the machine value. The `/crew:init` template writes every key except the personal ones, so it does not shadow your machine defaults for those; a repo `/crew:init` set up before that still spells `autopilot.mode: "off"`, which holds a machine `plan` down until you remove it (`--explain --all` names it as a `shadow:`).

Seeing What Is in Force

- `/crew:config --show` prints every machine-settable key, its value and the layer it came from, and names any key that takes effect nowhere.
- `/crew:config` with no argument is a menu that writes either file through the validated path. It shows a dry run first, refuses unknown keys and out-of-range values, and marks a widening with `!`.
- From a shell: `python3 plugin/crew/hooks/scripts/crew_config.py --explain` is the same table, `--explain --all` adds every repo-only key and names each shadowing repo value, and `--models` is the per-role provider table.

Backups and Rebuilding a Lost Config

Every write a crew script makes to either file first copies the old bytes to `~/.claude/crew/backups/` (the newest 20 per file kept) and is refused when that copy fails; a hand edit is not backed up. `crew_config.py --backups` lists the machine file's (add `--repo` for the repo file's) and `--restore <stamp> --apply` puts one back. Your non-default values are kept in `~/.claude/crew/profile.json` (and in your vault when `memory.vaultPath` is set), so `--rebuild --repo` or `--rebuild --global` regenerates a lost or corrupt file from the template plus that profile, as a dry run until `--apply`.

Common Setups

- **Personal defaults once, for every repo.** Put providers, models, `notify.*`, the guards and the `autopilot.*` keys in `~/.claude/crew/config.json`. Leave repo files to repo facts.
- **Self-approval under autopilot.** Set `autopilot.mode: "plan"` and choose `autopilot.approval` (`human`, `self` or `risk`) in either

file, and set `scope.allowCliApproval: true` in the repo file, which is the only file that key is read from. Review acceptance and production deploys still stop for you.

- **Notifications.** In the machine file, set `notify.provider` and the name of the environment variable holding the token in `notify.tokenEnv`. The secret stays in your environment, never in the file. `tokenEnv` is machine-only: a repo file's value is ignored with a notice.

The key list is `plugin/crew/CONFIG.md`, which ships with the plugin; the reasoning behind the keys is in `docs/crew/config-rationale.md`.

The Human-Only Opt-Out (`humanOnly`)

`humanOnly.approve` and `humanOnly.promoteMarker` are the owner's opt-out of two human-only rules: `/crew:approve`, and the production promote marker `.crew/.approved-<env>-<sha>`. `null` and `true` mean human-only. A `false` takes effect only when all of these hold (`crew_humanonly.resolve`):

- it is in the machine file, or in a repo `.crew/config.json` that git does not track (a tracked or committed repo file can never opt out; "could not tell" counts as tracked);
- neither layer says `true` (a `true` anywhere vetoes, the stricter wins);
- it was written by a double confirmation: `crew_confirm_config.py propose` records the change and prints an id with two lines the agent relays to you word for word (the change, and the exact command; crew 1.3.70), and the owner types `/crew:confirm-config <change_id>` twice, in two prompts, within 15 minutes. `crew_config.py --set` and `/crew:config` refuse a `false` (and a repo `null` or removal, which drops a veto); a `false` written by hand reads as human-only.

What is live (crew 1.3.33). The prompt hook (`approval_hook.py`) now records `/crew:confirm-config <change_id>` from a prompt you typed, so a double confirmation can complete, and `humanOnly.approve: false` takes effect through ONE decision point, `crew_ticket.cli_approval_allowed`: with an opt-out in force a `cli`-recorded approval receipt counts as if `scope.allowCliApproval` were `true`, for the scope guard, the Stop audit and every autopilot reader at once. It does NOT let the session run `crew_ticket.py approve` (the scope guard still refuses that text); the session's route is `crew_autopilot.py approve`, which still needs `autopilot.approval self` or `risk` and `autopilot.mode`

`plan` or `backlog`. Anything unclear - the module missing, the resolver raising or answering anything but exactly `humanOnly: false` - reads as human-only.

Finding the change id (crew 1.3.69). Type `/crew:confirm-config` with no id (or with an id that is wrong, expired or already used) and the hook refuses it, records nothing, and lists the proposals in this machine's store (`~/.claude/crew/confirm-config/`): each pending one's id, key, current value `->` `false`, layer and file, age, time left and confirmations so far, then up to five from the last day that can no longer be confirmed. With none pending it says so, and why one may be missing: none was made, it expired, it was already applied, cancelled or refused, or it was proposed under another home directory (WSL rather than Windows has its own store). The store is per machine and home directory, not per repository, so a proposal made in one repo or worktree is listed from any other. The listing never confirms: even with exactly one pending proposal a bare `/crew:confirm-config` records nothing, because "the only pending one" is a choice whoever ran `propose` makes, and the id you type is what binds the change. A store crew cannot read is reported as "could not tell", never as none pending.

Still human-only (crew 1.3.33). No hook reads `humanOnly.promoteMarker`: the promote marker is human-only by instruction alone, and no hook checks who created it. `crew_confirm_config.py resolve` shows what each key would resolve to.

Who can write the receipts. The double confirmation is recorded only by the prompt hook, from your own prompt. The scope guard (when `scope.mode` is not `off`) refuses every Write/Edit under `~/.claude/crew/confirm-config/` and every shell command that names that store beside a writing word. That is a textual tripwire, not a sandbox: a session that sets out to forge a receipt (`python -c`, an encoded command, a variable holding the path) still can, and `scope.mode: off` has no tripwire at all. Treat `humanOnly` opt-outs as meaningful only with `scope.mode block`.

`/crew:approve-mass <id>` (crew 1.3.36). Your yes to ONE blocked mass operation, typed by you and recorded by the same prompt hook in `<git-common-dir>/crew/mass-approvals/<id>.json`, good for 15 minutes. The directory is under `<git-common-dir>/crew/`, which the scope guard refuses to every Write/Edit and every shell command naming it, so the session cannot write the file. The cloud guard (`cloud_guard.py`) prints the `<id>` with its stop and reads the file; see "Mass operations".

Mass Operations (`guards.mass`, Crew 1.3.36)

The rule for an autopilot that runs without a human: full automation, except a mass delete, a mass create or a mass change, which wait for your yes. The stop is armed by `guards.mass.enabled: true` (either layer) or by a double-confirmed `humanOnly` opt-out, and by nothing else; it is off by default. It rides on the cloud guard's hook, judges every shell command, and works even where `guards.cloudGuard` is `off`.

Limit	Default	Counts
`deleteCount`	10	files a command removes
`createCount`	20	files it adds
`changeCount`	30	files it rewrites

A count AT the limit stops. The defaults were measured on this repo's own history (104 days, 623 merged PRs: a PR adds a median 0 and p95 16 files, changes a median 5 and p95 30, deletes none in all but 15). `guards.mass.overrides.<files|git|terraform>.<limit>` replaces one limit for one kind of command; with nothing set `terraform` is delete 3, create 10, change 10 (inferred - the history held no terraform). The machine file sets the limits and `guards.mass.exempt`; the repo file can lower a limit and turn the stop on (`enabled` is on if either layer says so), so a cloned repo can tighten the stop and never loosen yours. `guards.mass.exempt` (machine file only) lists repo-relative globs of generated files that never count; node_modules, `__pycache__`, `.venv`, caches, `build`, `dist`, `.work` and worktrees (including `~/.claude/crew/worktrees/<repo>/<name>`) are always exempt.

What it sees: `rm`, `Remove-Item`, `git clean -f`, `git rm`, `find -delete`, `rsync --delete` (delete); `cp`, `mv`, `rsync`, `Copy-Item`, `touch` (create and change); `sed -i` and `git reset --hard` / `checkout -- <paths>` / `restore` (change); `git commit`, `git push` and `gh pr create`, which count the added, deleted and changed files of the commit or of the whole branch against its base - so Write and Edit work is counted where it is about to be shared; `terraform apply <plan>` (the plan's own changes), `destroy` and `apply -auto-approve`; and an unfiltered SQL `DELETE` or `UPDATE`.

A count that cannot be taken - a variable in a path, a path after a `cd`, `find -delete`, `xargs rm`, a failed `git`, an unfiltered SQL statement (no

command text says how many rows it hits) - is "could not tell", and could not tell stops. It does NOT see a script (`bash cleanup.sh`, `python -c`, `make clean`), an MCP server, the Write and Edit tools before they are committed, `tar -x` or `unzip`, `git merge` or `pull`, or many small commands each under a limit: a tripwire in front of native permissions, not a sandbox.

A stopped command prints an id. Attended, the stop is an `ask`; unattended it is a `deny`. Type `/crew:approve-mass <id>` in the session, then re-run the exact command. One id covers every limit that command trips.

Git Commands (`guards.git.allow`, Crew 1.3.75)

The owner's switch that stops crew's command guards from blocking or asking on git commands, destructive ones included. Off by default (`null`, or `"off"`).

Value	What passes
`null` / `"off"`	nothing changes: every guard judges git as before
`"all"`	every git command
`["commit", "push", ...]`	a git command whose subcommand is listed; every other git command keeps today's checks

A listed subcommand passes with every option it takes: listing `push` also lets `git push --force` and `git push origin +main` through, because the force-push rule is a finding on the `push` subcommand. To keep force pushes stopped, leave `push` out of the list.

What it loosens, and nothing else: the cloud guard's `guards.forcePush` rule (`git push --force`, `+ref`; not `xargs git`, whose appended arguments crew cannot see); the mass stop's git counts (`git commit`, `git push`, `git clean -f`, `git rm`, `git reset --hard`, `git checkout -f` / `-- <paths>`, `git restore`); and the promote gate for a command line made ONLY of covered git commands, so a commit message or heredoc that merely mentions a declared deploy no longer trips it. Everything else on the

same line is judged as before (`git commit && rm -rf big/` still meets the mass stop for the `rm`). A declared deploy that is itself a git command (`git push prod main`) stays gated: that is a deploy, not a git block. `gh` is not git: `gh pr merge --admin` (`guards.adminMerge`), `gh pr create`'s mass count, workflow dispatches and `gh api` calls are untouched.

Who can turn it on: the machine file only, and only through the double confirmation. `crew_confirm_config.py propose --layer machine --key guards.git.allow --value '"all"'` (or `'["commit","push"]'`) records the change and prints an id; you type `/crew:confirm-config <change_id>` twice, in two prompts, within 15 minutes. A value counts only while the newest settled receipt for that file is a whole double confirmation (two confirmations from two distinct prompts you typed, inside the proposal's window, for that file) of exactly that value, AND the machine file is still the file that confirmation left: the same bytes, size, modification time and change time (the metadata-change time: POSIX `ctime`, NTFS ChangeTime on Windows). crew's own config writer (`crew_config.py --set`, `/crew:config`, a confirmed `humanOnly` change) carries the receipt over when it changes another key. Any other change to the machine file turns the switch off until you confirm it again: a hand edit, an agent's Edit, a restore from backup, even setting `"off"` by hand and then the same `"all"` back, or putting a copy of the confirmed file back with `cp -p` (bytes and modification time restored; the change time still moves). One gap, on Windows only: a copy made AND put back with a Windows file copy (`Copy-Item`, Explorer, `shutil.copy2`) carries the change time over with the bytes, so crew cannot tell it from the confirmed file. `crew_config.py --set` and `/crew:config` refuse every value but `"off"` and null, and a crew write back to `"off"` revokes the receipt. A repo file can never turn it on; a repo `"off"`, a malformed repo value, or a repo `.crew/config.json` crew cannot read or parse (a link that does not resolve, or a `.crew` that is a file, included) vetoes it. Anything crew cannot read - a value outside the table, a store it cannot read, a line it cannot split, a subcommand it cannot see - is off.

Never exempt, whatever the value: a git command that hands git config or a command of its own - a global `-c` (an alias `!cmd`, `core.sshCommand`, `core.pager`), `--config-env` or `--exec-path`, a `GIT_*` environment variable set on the line (or assigned and then exported by name, `X=v; export X`), one git or what it starts reads a program or config from (`HOME`, `XDG_CONFIG_HOME`, `PATH`, `EDITOR`, `PAGER`, `SSH_ASKPASS`, `LD_PRELOAD`, ...), or an option or subcommand that runs a command it is given (`rebase --exec/-x`, `submodule foreach`, `bisect run`, `push/send-pack --receive-pack/--exec`, `fetch-pack --upload-pack/--exec`, `fetch/pull/clone/ls-remote --upload-pack`, `clone -u/-c/--template`, `grep -O`, `difftool --extcmd`, `filter-branch`, `instaweb`, `send-email --*-cmd`, `daemon --access-hook`), in any abbreviation

git accepts (`--exe`, `--receive=...`: git takes an unambiguous prefix of a long option). Nor a git command crew cannot read whole: a word built when it runs (`"$OPT"`, `${X:-...}`, `$env:X`, a backtick or `$(...)` substitution, a PowerShell `@splat`; in bash an unquoted brace expansion (`--rece{ive-pack,}=x`), glob (`*`, `?`, `[]`) or leading `~`; in PowerShell `--%`, a `%NAME%`, a glob character or a leading `~`) - so `git commit -m "$(git rev-parse HEAD)"` is judged as before - or a git command `xargs` / `parallel` runs, whose appended arguments come from input crew cannot see. In PowerShell a single `&` (the background or call operator) makes the line unreadable for the promote gate; `2>&1` and a quoted `&` do not. A git command aimed at another repository (`git -C <dir>`) passes only when that repository's own file does not veto it either; one whose repository crew cannot tell (`--git-dir`, `--work-tree`, a `-C` built from a variable, or any `cd` on the line) is judged as before.

What a receipt cannot prove: the store (`~/.claude/crew/confirm-config/`) is plain files. It is protected from an agent's writes ONLY while `scope.mode` is on: then the scope guard refuses a Write/Edit into it, and a shell command naming it beside a writing word. With `scope.mode` off nothing stops an agent writing a receipt, and even with it on a process that runs arbitrary code as you can write a whole, well-formed record. And `"all"` does not stop what git runs from its own persistent configuration: hooks (`.git/hooks/`, `core.hooksPath`), aliases (`alias.*` set in a config file earlier), and `core.sshCommand`, `core.pager`, `core.editor`, filters and the like all run under `"all"` exactly as they would with crew absent. crew reads only the command line, never git's config files or hooks.

`python3 hooks/scripts/crew_gitallow.py --root .` prints what is in force. Each pass is logged to `.crew/guard.log` with the policy `gitAllow:<value>`.

Reference (Generated)

Generated from the code by `python3 docs/guides/crew/src/config_reference.py --write`. Do not edit by hand: `python3 scripts/check-marketplace.py` fails when this file is stale.

107 keys: 69 settable in the machine-global file, 38 repo-only.

Columns:

- **Layer**: which file may set the key.
- **Default**: what `default_config()` returns. Where the machine default differs, both are shown.

- **Values**: the values crew accepts, read from the same tuple the validator reads. "not validated" means crew checks nothing and names the file that reads the key.
- **Since**: the first crew version whose template declared the key. Keys in the first template (crew 0.11.0) read "0.11.0 or earlier".

Layers

- **repo**: only `.crew/config.json` may set it; the global file's value is pruned on read and refused on write.
- **both**: either file may set it; the repo value wins over the machine one.
- **both, ratchet**: either file may set it, and the NARROWER of the two wins (`crew_guards.RATCHETED_KEYS`). A repo can tighten what the machine allows, never loosen it.
- **both, widening warned**: either file may set it and the repo wins, but a write that widens it is flagged (`crew_config._RATCHETED`).
- **both, stricter wins**: a personal key (`crew_guards.PERSONAL_KEYS`): either file may set it, and where both do the STRICTER value wins; a layer that is silent (absent or `null`) imposes nothing, so a machine value is your default for every repo and a repo may only narrow it.
- **machine-arms**: only the global file can turn it on (exactly `true`); a repo may only veto it with `false` (`crew_config.REPO_VETO_ONLY`).
- **machine-only**: read from the global file alone; a repo's own value is never consulted.
- **human-only opt-out**: a `humanOnly` key (`crew_humanonly`): `false` opts out only from the machine file or an untracked repo file, only after the owner types `/crew:confirm-config <change_id>` twice, and a `true` in either layer vetoes it. Not acted on by any hook yet (see the section below).

Top-Level Keys

Setting	Layer	Default	Values	Since	Summar
`schema`	repo	`8`	not validated - read by `plugin/crew/hooks/scripts/crew_state.py` (expects integer)	0.11.0 or earlier	Config sc version; `/crew:mig` upgrades one.
`tracker`	repo	`"files"`	`files` | `obsidian` | `jira` | `sdp`	0.11.0 or earlier	Where tic `crew_trac` answers c `tell` whe configs d
`verifyGate`	repo	`true`	not validated - read by `plugin/crew/hooks/scripts/verify-gate.sh` (expects boolean)	0.11.0 or earlier	Run the S gate.

Qa

Setting	Layer	Default	Values
`qa.provider`	both	`"auto"`	`auto` | `claude` | `codex` | `copilot` | `kimi`
`qa.order`	both	`["codex", "kimi", "copilot", "claude"]`	list of: `claude` | `codex` | `copilot` | `kimi`
`qa.fallback`	both	`"claude-sonnet-5"`	not validated - read by `plugin/crew/commands/review.md` (expects model id)

Setting	Layer	Default	Values
`qa.codex.model`	both	`null`	not validated - read by `plugin/crew/commands/review.md` (expects string or null)
`qa.codex.reasoningEffort`	both	`null`	`none` | `minimal` | `low` | `medium` | `high` | `xhigh` | `max` (listed in `plugin/crew/commands/review.md`; not validated)
`qa.copilot.model`	both	`null`	not validated - read by `plugin/crew/commands/review.md` (expects string or null)
`qa.kimi.model`	both	`null`	not validated - read by `plugin/crew/commands/review.md` (expects string or null)
`qa.roles`	both	`{}`	object of role pins; each pin's provider is checked (checked in `plugin/crew/hooks/scripts/crew_config.py`)

Dev

Setting	Layer	Default	Values
`dev.provider`	both	`"claude"`	`claude` | `codex` | `copilot` | `kimi`
`dev.fallback`	both	`"claude-sonnet-5"`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects model id)

Setting	Layer	Default	Values	
`dev.codex.model`	both	`null`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string or null)	
`dev.codex.reasoningEffort`	both	`null`	`none` | `minimal` | `low` | `medium` | `high` | `xhigh` | `max` (listed in `plugin/crew/commands/review.md`; not validated)	
`dev.copilot.model`	both	`null`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string or null)	
`dev.kimi.model`	both	`null`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string or null)	
`dev.roles`	both	`{}`	object of role pins; each pin's provider is checked (checked in `plugin/crew/hooks/scripts/crew_config.py`)	

`secondOpinion`

Setting	Layer	Default	Values	Since	Summa
`secondOpinion.provider`	both	`"none"`	not validated - read by `plugin/crew/commands/plan.md` (expects string)	0.11.0 or earlier	Second-opinion provider

Setting	Layer	Default	Values	Since	Summa
					for plans `none` is o
`secondOpinion.mode`	both	`"cli"`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string)	0.11.0 or earlier	How the second opinion reached.
`secondOpinion.model`	both	`null`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string or null)	0.11.0 or earlier	Second-opinion model.

Jira

Setting	Layer	Default	Values	Since	Summary
`jira.project`	repo	`null`	not validated - read by `plugin/crew/commands/jira-sync.md` (expects string or null)	0.11.0 or earlier	Jira project key. No consumer found (docs/crew/config-rationale.md).
`jira.cloudId`	repo	`null`	not validated - read by `plugin/crew/commands/jira-sync.md` (expects string or null)	0.19.10	Jira cloud id, cached by `/crew:jira-sync`; read by nothing.

Obsidian

Setting	Layer	Default	Values	Since
`obsidian.vaultPath`	repo	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_tracker.py` (expects path or null)	0.11.0 or earlier
`obsidian.boardDir`	repo	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_tracker.py` (expects path or null)	0.11.0 or earlier

Memory

Setting	Layer	Default	Values	Since	Su
`memory.mode`	both	`"repo"`	not validated - read by `plugin/crew/skills/crew-memory/SKILL.md` (expects string)	0.11.0 or earlier	W m liv or va
`memory.vaultPath`	both	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_recall.py` (expects path or null)	0.11.0 or earlier	Th O va fo m

Context

Setting	Layer	Default	Values
`context.enabled`	repo	`true`	not validated - read by `plugin/crew/hooks/scripts/context` (expects boolean)
`context.warnAt`	repo	`0.4`	not validated - read by `plugin/crew/hooks/scripts/context` (expects number)
`context.reserveTokens`	repo	`0`	not validated - read by `plugin/crew/hooks/scripts/context` (expects integer or null)
`context.autoClear.enabled`	machine-arms	`null`	`null` | `true` | `false` (checked in `plugin/crew/hooks/scripts/crew_au`

Setting	Layer	Default	Values
`context.autoClear.method`	both	`"auto"`	`auto` | `none` | `notify` | `tmux` | `xdotool` | OS: linux: auto, none, notify, tmux, x wtype; macos: auto, none, notify, tm windows: auto, none, notify; windov auto, none, notify, tmux
`context.autoClear.delaySeconds`	both	`3`	number (coerced in `plugin/crew/hooks/scripts/crew_au`
`context.autoClear.minHandoffLines`	both	`5`	number (coerced in `plugin/crew/hooks/scripts/crew_au`
`context.autoClear.onlyRepos`	machine-only	`null`	list of absolute repo paths, or null (c `plugin/crew/hooks/scripts/crew_au`

Setting	Layer	Default	Values
`context.autoClear.onlySessions`	machine-only	`null`	list of session ids, or null (coerced in `plugin/crew/hooks/scripts/crew_au`
`context.autoClear.wrapUp`	machine-arms	`null`	`null` | `true` | `false` (checked in `plugin/crew/hooks/scripts/crew_au`

Setting	Layer	Default	Values

Resume

Setting	Layer	Default	Values	Since	Sur
`resume.auto`	machine-arms	`null`	`null` | `true` | `false` (checked in `plugin/crew/hooks/scripts/crew_resume.py`)	1.0.40	Aut resu afte /cl a m /co Onl ma file arm (exa tru rep fal vet

humanOnly

Setting	Layer	Default	Values
`humanOnly.approve`	human-only opt-out	`null`	`null` | `true` | `false` (checked in `plugin/crew/hooks/scripts/crew_humanonly.py`)

Setting	Layer	Default	Values
`humanOnly.promoteMarker`	human-only opt-out	`null`	`null` | `true` | `false` (checked in `plugin/crew/hooks/scripts/crew_humanonly.py`)

Worktrees

Setting	Layer	Default	Values	Since	S
`worktrees.root`	both	`null`	absolute path or null (coerced in `plugin/crew/hooks/scripts/crew_worktrees.py`)	1.3.19	T

Setting	Layer	Default	Values	Since	S

Promote

Setting	Layer	Default	Values	S
`promote.deadlineSeconds`	both	`null`	number of seconds, or null (coerced in `plugin/crew/hooks/scripts/_promote_gate.py`)	1

Setting	Layer	Default	Values	S

Reaper

Setting	Layer	Default	Values	S
`reaper.enabled`	both	`false`	boolean (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1

Setting	Layer	Default	Values	S
`reaper.dryRun`	both	`true`	boolean (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1
`reaper.spinCpuPct`	both	`50`	percent of one core (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1
`reaper.spinMinAgeMin`	both	`15`	number of minutes (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1

Setting	Layer	Default	Values	S
`reaper.blockedMinAgeHours`	both	`2`	number of hours (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1
`reaper.protectedRegex`	both	`null`	regular expression, or null (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1
`reaper.intervalMin`	both	`10`	whole number of minutes (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1
`reaper.onSessionStart`	both	`false`	boolean (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1

Emergency

Setting	Layer	Default	Values	S
`emergency.standDown`	repo	`true`	not validated - read by `plugin/crew/hooks/scripts/_common.sh` (expects boolean)	C e
`emergency.maxTtlMinutes`	repo	`480`	integer (coerced in `plugin/crew/hooks/scripts/crew_incident.py`)	C e

Notify

Setting	Layer	Default	Values	Sinc
`notify.provider`	both	`null`	telegram, teams, none or null; any other value sends nothing (coerced in `plugin/crew/hooks/scripts/crew_notify.py`)	0.11. or earli

Setting	Layer	Default	Values	Sinc
`notify.tokenEnv`	machine-only	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_notify.py` (expects string or null)	0.11. or earli
`notify.chatId`	both	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_notify.py` (expects string or null)	0.11. or earli
`notify.events`	both	`["blocker", "deploy", "question"]`	list of event names; an unknown name is dropped with a notice (coerced in `plugin/crew/hooks/scripts/crew_notify.py`)	0.11. or earli

Setting	Layer	Default	Values	Sinc

Platform

Setting	Layer	Default	Values	Si
`platform.os`	repo	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_platform.py` (expects string or null)	0.1 or ea
`platform.wsl`	repo	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_platform.py` (expects boolean or null)	0.1 or ea
`platform.shell`	repo	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_platform.py` (expects string or null)	0.1 or ea
`platform.windowsHostIp`	repo	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_platform.py` (expects string or null)	0.1 or ea

Pm

Setting	Layer	Default	Values	Since
`pm.authority`	both, widening warned	`"report-only"`	`report-only` | `act` | `autonomous`	0.11.0 or earlier
`pm.maxDispatches`	both	`3`	integer (coerced in `plugin/crew/hooks/scripts/crew_state.py`)	0.11.0 or earlier

Graph

Setting	Layer	Default	Values	Since	Summa
`graph.out`	repo	`"graphify-out"`	not validated - read by `plugin/crew/hooks/scripts/crew_state.py` (expects path)	0.11.0 or earlier	Where t code graph is written.

Install

Setting	Layer	Default	Values	Since	Summary
`install.policy`	both, ratchet	`"manual"`	`manual` | `ask` | `auto` (ratchet: narrower layer wins; listed narrowest first)	0.19.18	Whether crew may install a missing prerequisite.

Guards

Setting	Layer	Default	Values
`guards.profile`	both, ratchet	`"off"`	`off` | `standard` | `strict`
`guards.overrides`	both, ratchet	`{}`	object of guard name to tier (checked in `plugin/crew/hooks/scripts/crew_guards.py`)
`guards.mass.enabled`	both	`false`	boolean (coerced in `plugin/crew/hooks/scripts/crew_mass.py`)
`guards.mass.deleteCount`	both	`10`	integer (1 or more) (coerced in `plugin/crew/hooks/scripts/crew_mass.py`)
`guards.mass.createCount`	both	`20`	integer (1 or more) (coerced in `plugin/crew/hooks/scripts/crew_mass.py`)
`guards.mass.changeCount`	both	`30`	integer (1 or more) (coerced in `plugin/crew/hooks/scripts/crew_mass.py`)
`guards.mass.overrides`	both	`{}`	object of category to {deleteCount, createCount, changeCount} (checked in `plugin/crew/hooks/scripts/crew_mass.py`)

Setting	Layer	Default	Values
`guards.mass.exempt`	both	`[]`	list of globs (coerced in `plugin/crew/hooks/scripts/crew_mass.py`)
`guards.git.allow`	both	`null`	null, `"off"`, `"all"` or a list of git subcommand names (coerced in `plugin/crew/hooks/scripts/crew_gitallow.py`)

Production

Setting	Layer	Default	Values	Since
`production.databases`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/crew_config.py` (expects list of globs)	0.19.30
`production.hosts`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/crew_config.py` (expects list of globs)	0.19.30

Cloud

Setting	Layer	Default	Values	Si
`cloud.awsProfiles`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/cloud_guard.py` (expects list of names)	1.0
`cloud.awsRegions`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/cloud_guard.py` (expects list of names)	1.0
`cloud.azureSubscriptions`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/cloud_guard.py` (expects list of names)	1.0

Environments

Setting	Layer	Default	Values
`environments.nonProd`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/crew_config.py` (expects list of globs)
`environments.prodUnattended`	both, ratchet	`false`	`false` | `true` (ratchet: narrower layer wins; listed narrowest first)

Setting	Layer	Default	Values
`environments.workflows`	repo	`{}`	not validated - read by `plugin/crew/hooks/scripts/crew_config.py` (expects object of glob to string)
`environments.preProduction`	repo	`{}`	not validated - read by `plugin/crew/hooks/scripts/crew_preprod.py` (expects object of exact name to {reason, targets})

Change

Setting	Layer	Default	Values	Since	Summary
`change.requireForProduction`	both, ratchet	`false`	`true` | `false` (ratchet: narrower layer wins;	0.19.31	Require an approved change request before `/crew:promote`

Setting	Layer	Default	Values	Since	Summary
			listed narrowest first)		`production`: a repo may turn it on, never off.

Git

Setting	Layer	Default	Values	Sin
`git.forbiddenTrailers`	both	`[]`	list of trailer tokens (letters, digits and -, no :) (checked in `plugin/crew/hooks/scripts/crew_trailers.py`)	1.0

Scope

Setting	Layer	Default	Values	Sin
`scope.mode`	repo	`"off"`	`off` | `report` | `block` | `auto`	1.0.2
`scope.allowCliApproval`	repo	`false`	`false` | `true` (checked in `plugin/crew/hooks/scripts/crew_ticket.py`)	1.0.2

Autopilot

Setting	Layer	Default	Values
`autopilot.mode`	both, stricter wins	`"off"`	`off` | `plan` | `backlog` (checked in `plugin/crew/hooks/scripts/crew_autopilot` personal: listed strictest first, the stricter wi

Setting	Layer	Default	Values
`autopilot.maxPhases`	both, stricter wins	`12`	positive integer (checked in `plugin/crew/hooks/scripts/crew_autopilo` personal: the smaller wins
`autopilot.deploy`	both, stricter wins	`"none"`	`none` | `nonprod` | `all`; personal: listed strictes stricter wins
`autopilot.approval`	both, stricter wins	`"risk"`	`human` | `risk` | `self`; personal: listed strictest stricter wins
`autopilot.questions`	both, stricter wins	`"risk"`	`human` | `risk` | `self`; personal: listed strictest stricter wins
`autopilot.maxAutoReplans`	repo	`0`	non-negative integer (checked in `plugin/crew/hooks/scripts/crew_autopilo`

Setting	Layer	Default	Values
`autopilot.ship`	repo	`"merge"`	`pr` | `merge`
`autopilot.sleep.schedule`	repo	`null`	HH:MM-HH:MM or null (checked in `plugin/crew/hooks/scripts/crew_sleep.py`)
`autopilot.sleep.approval`	repo	`null`	`null` | `human` | `self` | `risk` (checked in `plugin/crew/hooks/scripts/crew_sleep.py`)
`autopilot.sleep.questions`	repo	`null`	`null` | `human` | `self` | `risk` (checked in `plugin/crew/hooks/scripts/crew_sleep.py`)
`autopilot.sleep.deploy`	repo	`null`	`null` | `nonprod` | `none` (checked in `plugin/crew/hooks/scripts/crew_sleep.py`)

Setting	Layer	Default	Values
`autopilot.maxTicketsPerRun`	repo	`3`	positive integer (checked in `plugin/crew/hooks/scripts/crew_autopilot`
`autopilot.maxTokensPerSession`	repo	`2000000`	positive integer (checked in `plugin/crew/hooks/scripts/crew_autopilot`
`autopilot.maxLanes`	repo	`null`	positive integer or null (checked in `plugin/crew/hooks/scripts/crew_wave.py`)

Setting	Layer	Default	Values
`autopilot.reviewPolicy`	repo	`"stop"`	`stop` | `clean-only` | `fix-and-rereview`

Tickets

Setting	Layer	Default	Values	Since
`tickets.baseBranch`	repo	`null`	branch name or null (checked in `plugin/crew/hooks/scripts/scope_base.py`)	1.0.158

Setting	Layer	Default	Values	Since

`Route`

Setting	Layer	Default	Values	Since	Summ
`route.enabled`	both	`false`	`false` | `true` (checked in `plugin/crew/hooks/scripts/crew_route.py`)	1.0.42	Route plain- prom `/crew` comm only t JSON `true` a it.

`unattendedCloud`

Setting	Layer	Default	Values
`unattendedCloud.aws.readOnly.profile`	machine-only	`null`	profile name, or null (coerced in `plugin/crew/hooks/scripts/cre`

Setting	Layer	Default	Values
`unattendedCloud.aws.readOnly.identity`	machine-only	`null`	ARN prefix ending in `/`, or null (`plugin/crew/hooks/scripts/cre`
`unattendedCloud.aws.readOnly.region`	machine-only	`null`	region, or null (coerced in `plugin/crew/hooks/scripts/cre`
`unattendedCloud.aws.nonProd`	machine-only	`{}`	None (checked in `plugin/crew/hooks/scripts/cre`

Coming (Not in Code Yet)

Keys from approved tickets that have not landed. Each moves into the table above when its ticket lands, because it is then in the code; a test fails until it does (`test_no_coming_key_is_in_code`).

T-0030

Setting	Change	Layer	Default	Values	Summary
`coord.ttlMinutes`	new key	set when T-0030 lands	30		Lifetime of a cross-session coordination claim (1-10080).
`coord.channel`	new key	set when T-0030 lands	set when T-0030 lands		Where sessions coordinate.

T-0050

Setting	Change	Layer	Default	Values	Summary
`scope.allowCliApproval`	changes layer	both, stricter wins	false		Becomes settable in the glob the stricter layer winning, on reader (`crew_ticket.cli_approval_a` reads the global layer in a ha only follow-up to T-0050.